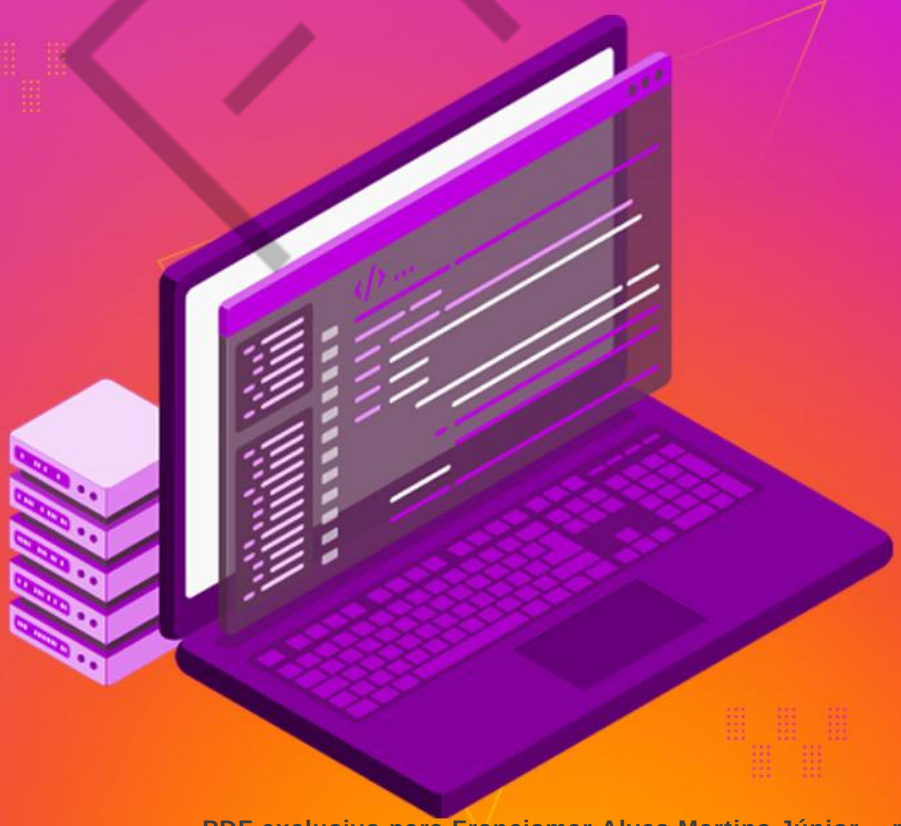
FIAP ON

IA COMO FERTILIZANTE DIGITAL - UM NOVO AGRONEGÓCIO DO AMANHÃ

ALGORITMOS GENÉTICOS: INTELIGÊNCIA INSPIRADA NA NATUREZA



4

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE FIGURAS

Figura 1 - Representação do DNA 6
Figura 2 - Estrutura de um algoritmo genético 7
Figura 3 - Algoritmos que combinam estratégias de exploration e exploitation.....10
Figura 4 - Ilustração que nos lembra de balancear estratégias de exploration e exploitation 12
Figura 5 - Passos do algoritmo genético 17
Figura 6 - Exemplo do crossover de ponto único 23
Figura 7 - Exemplo do crossover de ponto duplo 23
Figura 8 - Mutação de inversão.....24
Figura 9 - Mutação binária 25
Figura 10 - Mutação por permutação 25
Figura 11 - Ilustração do problema da mochila binária.....29
Figura 12 - Saída do Código-fonte 1 31
Figura 13 - Saída do Código-fonte 2 32
Figura 14 - Saída do Código-fonte 5 36
Figura 15 - Saída do Código-fonte 6 37
Figura 16 - Saída do Código-fonte 7 38

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE QUADROS

Quadro 1 - Vantagens e limitações dos algoritmos genéticos.....28

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 - Imports e carregamento de dados 30
Código-fonte 2 - Gerando a população 31
Código-fonte 3 - Definição de funções estruturantes do algoritmo genético 34
Código-fonte 4 - Função principal do algoritmo genético 35
Código-fonte 5 - Executando nosso algoritmo genético 35
Código-fonte 6 - Exibindo o melhor resultado encontrado 37
Código-fonte 7 - Evolução do resultado ao longo das gerações 37

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

SUMÁRIO

1 ALGORITMOS GENÉTICOS: INTELIGÊNCIA INSPIRADA NA NATUREZA 6
1.1 O que são algoritmos genéticos? 6
1.2 Aplicações 7
2 FUNDAMENTOS.....10
2.1 Exploration vs. Exploitation 10
2.2 Importância da aleatoriedade 12
2.3 Conceitos centrais 13
2.3.1 População 13
2.3.2 Cromossomo 14
2.3.3 Gene.....14
2.3.4 *Fitness function* 14
2.3.5 Conceitos fundamentais aplicados a um exemplo 15
3 CICLO DE EXECUÇÃO DE UM ALGORITMO GENÉTICO.....17
3.1 Etapas básicas 17
3.1.1 Inicialização aleatória da população.....18
3.1.2 Avaliação de aptidão (fitness) 18
3.1.3 Seleção dos indivíduos mais aptos 19
3.1.4 Reprodução (crossover e mutação) 19
3.1.5 Substituição para formar nova geração.....19
3.1.6 Critério de parada.....19
3.2 Um exemplo elucidativo 20
4 PONTOS DE ALEATORIEDADE 22
4.1 Seleção 22
4.2 Cruzamento (Crossover) 23
4.3 Mutação.....24
4.4 Parâmetros.....26
5 VANTAGENS E LIMITAÇÕES 27
6 HANDS ON: PRATICANDO ALGORITMOS GENÉTICOS 29
CONCLUSÃO.....39
REFERÊNCIAS.....40
GLOSSÁRIO 41

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1 ALGORITMOS GENÉTICOS: INTELIGÊNCIA INSPIRADA NA NATUREZA

1.1 O que são algoritmos genéticos?

Os Algoritmos Genéticos (AGs) são uma classe de métodos de otimização inspirados diretamente nos mecanismos naturais da evolução biológica, ou seja, da evolução do DNA (nosso código genético). Assim como na natureza, onde os organismos mais adaptados ao ambiente tendem a sobreviver e se reproduzir, os algoritmos genéticos operam com populações de soluções candidatas que evoluem ao longo de sucessivas gerações. Cada solução é representada por um “cromossomo”, que pode ser considerado como uma codificação de um possível caminho para solucionar o problema em questão. Ao longo do processo evolutivo artificial, essas soluções passam por operações análogas àquelas observadas na biologia: seleção dos mais aptos, cruzamento genético (crossover) entre indivíduos e mutações aleatórias.

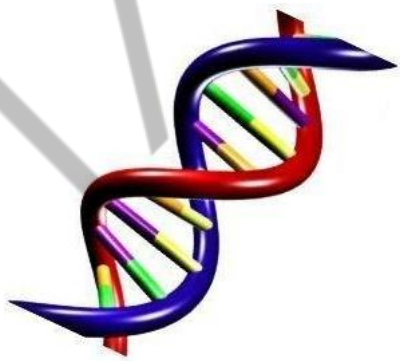


Figura 1 - Representação do DNA
Fonte: Xloptimizer (2025)

A essência dessa abordagem é simples, mas poderosa: manter a diversidade de soluções por meio da aleatoriedade enquanto refina as melhores soluções por meio da recombinação de bons indivíduos. Esse equilíbrio é fundamental para que o algoritmo possa escapar de soluções locais e buscar alternativas melhores em todo o espaço de possibilidades, o que o torna especialmente útil em problemas complexos, onde métodos determinísticos como gradiente descendente falham ou não são aplicáveis. A estrutura padrão de um algoritmo evolutivo está ilustrada na figura

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

“Estrutura de um algoritmo genético”. A seguir, vamos conhecer cada um destes passos e sua importância para o resultado.

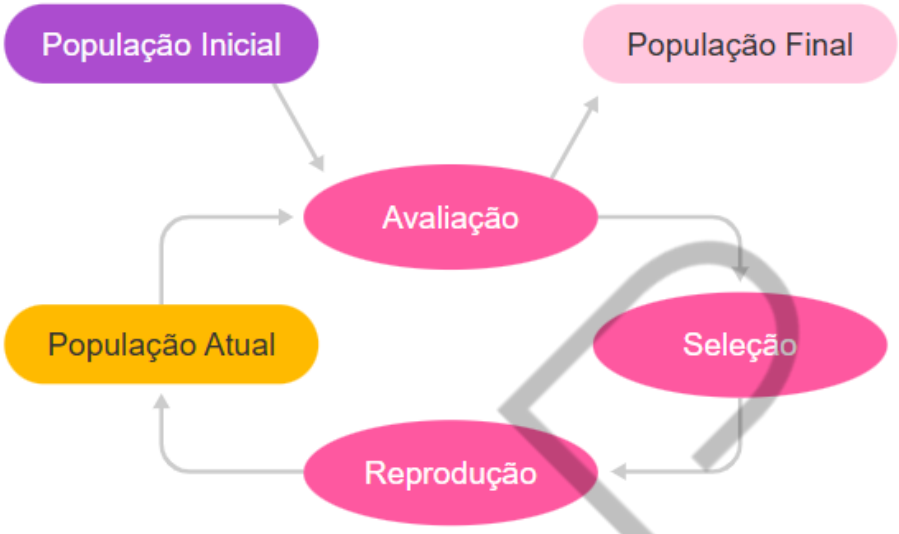


Figura 2 - Estrutura de um algoritmo genético
Fonte: Deviante (2021), adaptado por FIAP (2025)

Os algoritmos genéticos são classificados como métodos estocásticos e populacionais, ou seja, eles trabalham com múltiplas soluções simultaneamente, utilizando processos probabilísticos para guiar a evolução. Isso os diferencia de abordagens tradicionais baseadas em lógica determinística ou busca local. Devido à sua natureza adaptativa e à capacidade de explorar grandes espaços de busca com flexibilidade, os algoritmos genéticos são amplamente utilizados em áreas como engenharia, ciência de dados, biologia computacional, finanças e Inteligência Artificial. Ao combinar simplicidade estrutural com eficiência prática, eles representam uma ponte fascinante entre a biologia e a computação, trazendo para o mundo digital os princípios que regem a sobrevivência e adaptação dos organismos vivos.

1.2 Aplicações

Os algoritmos genéticos se destacam em cenários onde o espaço de busca é vasto, complexo e não possui uma estrutura matemática bem definida, como continuidade, linearidade ou derivabilidade. Em muitos desses casos, métodos tradicionais — como algoritmos baseados em gradientes — enfrentam dificuldades,

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

seja porque não conseguem explorar regiões distantes do espaço de busca, seja porque tendem a ficar presos em mínimos locais, resultando em soluções subótimas.

Os problemas de otimização real no mundo computacional frequentemente apresentam múltiplas soluções possíveis, interações entre variáveis altamente não lineares e objetivos conflitantes. Nessas situações, é essencial contar com uma abordagem que não apenas busque encontrar uma única solução, mas que seja capaz de explorar múltiplas alternativas simultaneamente. É justamente nesse contexto que os algoritmos genéticos brilham: ao trabalharem com uma população de soluções, eles têm a capacidade de explorar amplamente o espaço de busca, ao mesmo tempo em que aproveitam as melhores descobertas para guiar a geração seguinte.

Além disso, por serem baseados em operadores estocásticos como mutação e cruzamento, os algoritmos genéticos não seguem um caminho fixo rumo à solução, o que lhes confere maior robustez diante de superfícies de otimização irregulares e cheias de armadilhas locais. Essa capacidade de evitar soluções prematuras e manter uma diversidade de estratégias durante o processo de busca é um dos principais motivos pelos quais os algoritmos genéticos são tão valiosos em problemas onde outras abordagens simplesmente não funcionam ou fornecem resultados inconsistentes.

Veja algumas das principais áreas de aplicação dos algoritmos genéticos:

- **Problemas de otimização complexa onde métodos baseados em gradientes falham:**
  - Em funções altamente não lineares, com múltiplos mínimos locais ou que não possuem derivadas definidas, os AGs oferecem uma alternativa poderosa. Por não dependerem de derivadas, são ideais para problemas “caixa-preta”, em que se conhece apenas a saída de uma função, mas não sua estrutura interna.
- **Engenharia (projeto de circuitos, trajetórias de robôs, design de estruturas):**
  - Na engenharia, AGs são utilizados para projetar circuitos elétricos eficientes, otimizar trajetórias de movimento de robôs em ambientes complexos e encontrar estruturas mecânicas ou arquitetônicas mais

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

leves, resistentes ou baratas. Nessas aplicações, o espaço de busca é muitas vezes vasto, multidimensional e sujeito a múltiplas restrições físicas.

- **Inteligência Artificial (ajuste de hiperparâmetros, seleção de variáveis, evolução de arquiteturas de rede):**
  - Os AGs são ferramentas úteis para ajustar hiperparâmetros em modelos de Machine Learning, selecionar subconjuntos de variáveis mais relevantes para uma tarefa preditiva, e até mesmo para evoluir arquiteturas completas de Redes Neurais — área conhecida como neuroevolução. Essas tarefas, frequentemente combinatórias e complexas, se beneficiam da capacidade dos AGs de explorar soluções diversas.
- **Logística e roteamento (problema do caixeiro viajante, alocação de recursos, escalonamento):**
  - Em problemas como o clássico caixeiro viajante, em que se busca o caminho mais eficiente entre múltiplos pontos, os algoritmos genéticos conseguem gerar boas soluções aproximadas em tempo viável. Também são eficazes em desafios de alocação de recursos, planejamento de produção, escalonamento de tarefas e roteamento de veículos, onde há múltiplas restrições a serem consideradas.
- **Outros domínios (bioinformática, jogos, arte generativa, finanças):**
  - Na bioinformática, os AGs são aplicados na análise de sequências genéticas e modelagem de proteínas. Em jogos e arte generativa, permitem a criação automática de estratégias, personagens ou composições visuais inovadoras. No setor financeiro, são utilizados para otimizar carteiras de investimento, calibrar modelos de risco ou buscar padrões em séries temporais econômicas. Esses contextos compartilham a característica de apresentar espaços de busca altamente variáveis, nos quais os algoritmos genéticos conseguem navegar com flexibilidade.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

2 FUNDAMENTOS

2.1 Exploration vs. Exploitation

Nos algoritmos genéticos, assim como em diversas outras estratégias de otimização, dois conceitos fundamentais guiam o comportamento da busca por soluções: **exploration** (exploração) e **exploitation** (explotação). Entender o equilíbrio entre esses dois princípios é essencial para garantir que o algoritmo encontre soluções de boa qualidade sem se prender prematuramente a regiões subótimas do espaço de busca. Diversos algoritmos fazem combinações destas duas técnicas, como, por exemplo, os que estão ilustrados na figura a seguir:

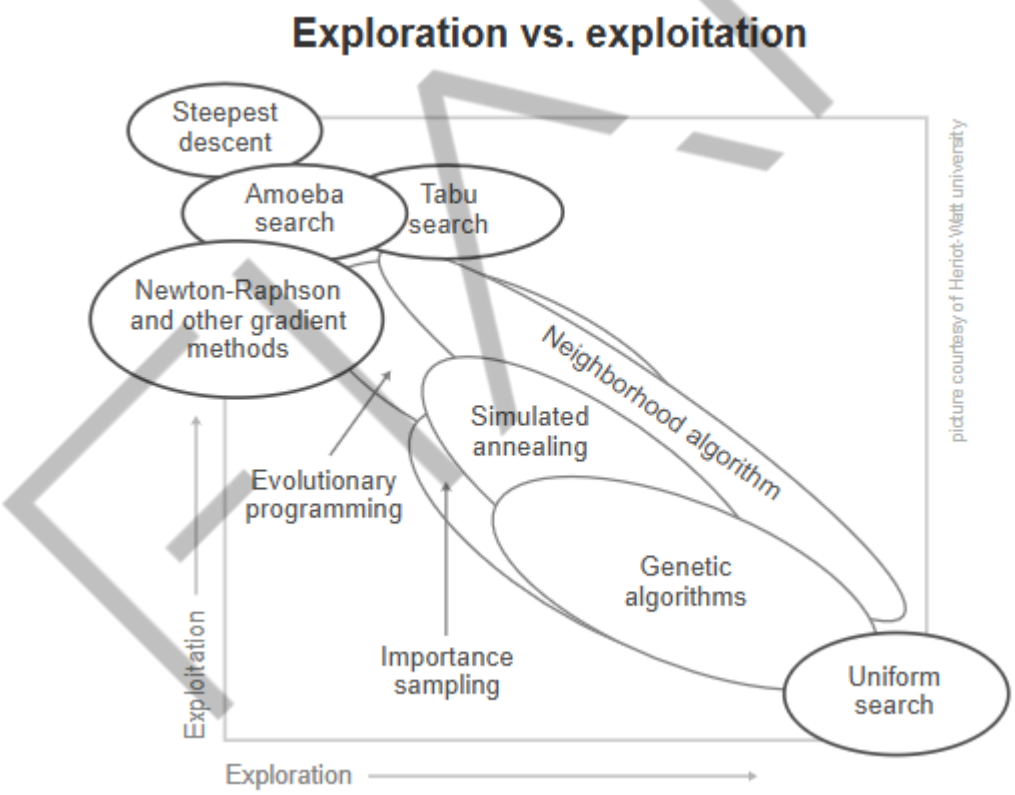


Figura 3 - Algoritmos que combinam estratégias de exploration e exploitation
Fonte: Géophysique (2016), adaptado por FIAP (2025)

A **exploration** refere-se à capacidade do algoritmo de investigar diferentes regiões do espaço de soluções, buscando novas possibilidades que talvez ainda não tenham sido testadas. É como lançar redes em múltiplas direções, tentando capturar boas ideias que estão distantes das soluções atuais. No contexto dos AGs, a

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

exploração é incentivada principalmente por mecanismos como a mutação, que introduz variações aleatórias nos cromossomos e mantém a diversidade genética da população. Por exemplo, ao aplicar mutações pontuais em soluções existentes, o algoritmo pode “descobrir” áreas promissoras do espaço que ainda não haviam sido exploradas, o que é especialmente útil em problemas com múltiplos ótimos locais.

Já a **exploitation** diz respeito ao uso mais intenso das melhores soluções já encontradas, aprofundando a busca em torno dessas regiões para refinar os resultados. Essa fase pode ser vista como o “aproveitamento” do conhecimento adquirido. Nos AGs, a exploração intensiva ocorre através de operadores como o crossover (cruzamento) e a seleção de indivíduos mais aptos, que favorecem a reprodução de soluções de alta qualidade. Por exemplo, quando os melhores cromossomos são selecionados repetidamente para cruzamento, espera-se que as próximas gerações herdem e combinem características vantajosas, otimizando ainda mais o desempenho da população.

O equilíbrio entre exploration e exploitation é o que determina o sucesso ou o fracasso do algoritmo. Se focarmos demais na exploration e negligenciarmos a exploitation, o processo pode se tornar aleatório e ineficiente, não se aproximando de uma boa solução. Por outro lado, uma exploração intensa sem diversidade pode causar convergência prematura, onde o algoritmo se fixa rapidamente em uma solução ruim e não consegue escapar. Por isso, bons algoritmos genéticos buscam manter a diversidade da população (exploration), ao mesmo tempo em que favorecem os indivíduos mais aptos (exploitation), garantindo uma busca global eficaz com refinamento local. Esse equilíbrio é um dos maiores trunfos dos algoritmos genéticos em ambientes complexos e incertos.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Figura 4 - Ilustração que nos lembra de balancear estratégias de exploration e exploitation
Fonte: Bizzdesign (2015)

2.2 Importância da aleatoriedade

A aleatoriedade é uma ferramenta poderosa nos algoritmos genéticos, mas seu uso exige reflexão e equilíbrio. A seguir, será listado alguns pontos que merecem atenção e reflexão:

- **Por que manter a aleatoriedade em um processo que busca soluções cada vez melhores?**
  - A resposta está na capacidade de explorar o espaço de busca de forma ampla e flexível. Ao introduzir diversidade na população inicial e ao aplicar mutações e cruzamentos aleatórios, o algoritmo evita se prender a regiões específicas e tem mais chances de descobrir soluções criativas e eficientes — especialmente em problemas com muitos ótimos locais.
- **Entretanto, a aleatoriedade não é uma panaceia.**
  - Embora seja essencial para escapar de armadilhas locais e fomentar a inovação, seu uso descontrolado pode ser tão prejudicial quanto a sua ausência. Um algoritmo com mutações excessivamente aleatórias, por exemplo, pode se comportar como uma busca cega, saltando de solução em solução sem nenhuma convergência significativa. Por outro lado, uma população inicial 100% aleatória, mas com operadores mal ajustados, pode rapidamente perder sua diversidade e convergir de forma prematura.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Portanto, **não basta empregar etapas aleatórias de qualquer forma**. É preciso que a aleatoriedade seja dosada e orientada: suficiente para manter a diversidade genética e permitir a descoberta de novas soluções, mas também limitada para que a população aprenda com os melhores indivíduos e refine seu desempenho. O verdadeiro valor da aleatoriedade está em sua capacidade de ser uma aliada da adaptabilidade, desde que seu uso possua uma intenção e esteja dentro de um processo cuidadosamente estruturado.

2.3 Conceitos centrais

A seguir, vamos explorar em detalhes os conceitos fundamentais que compõem os algoritmos genéticos. Entender como funcionam elementos como cromossomos, população, fitness, seleção, cruzamento e mutação é essencial para compreender a lógica por trás dessa poderosa estratégia evolutiva.

Esses componentes trabalham em conjunto para permitir que soluções evoluam ao longo do tempo, imitando o processo de seleção natural. Conhecê-los e compreendê-los é essencial para saber como adaptar seu problema para uma abordagem evolutiva.

2.3.1 População

Em algoritmos genéticos, **a população representa o conjunto de soluções candidatas que evoluem ao longo das iterações do algoritmo**. Cada **indivíduo** da população é uma possível resposta para o problema sendo resolvido, geralmente representado por um cromossomo, que codifica os parâmetros ou variáveis da solução. A diversidade da população é fundamental para garantir que o algoritmo explore diferentes regiões do espaço de busca, evitando que ele fique preso em soluções locais. A cada geração, a população passa por avaliações e transformações — como seleção, cruzamento e mutação — para formar uma nova população mais adaptada, aproximando-se progressivamente de soluções ótimas. Assim, a população é a base sobre a qual todo o processo evolutivo se desenvolve.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

2.3.2 Cromossomo

O cromossomo é a estrutura que codifica uma solução candidata dentro de um algoritmo genético. Ele representa, de forma compacta e manipulável, os parâmetros que definem uma possível resposta ao problema em questão. Cada indivíduo da população é composto por um único cromossomo, ou seja, **um indivíduo é uma solução, e seu cromossomo é a forma como essa solução está representada internamente**. A qualidade de um indivíduo, medida por sua aptidão (fitness), depende diretamente dos valores contidos em seu cromossomo. Durante o processo evolutivo, os cromossomos são cruzados e modificados, originando, assim, novos indivíduos que compõem a próxima geração.

2.3.3 Gene

O gene é a menor unidade de informação dentro de um cromossomo em um algoritmo genético. Ele representa um único parâmetro, característica ou variável da solução codificada. **Se pensarmos que o indivíduo é uma solução candidata ao problema e que o cromossomo é a representação completa dessa solução, então o gene é cada um dos componentes que formam esse cromossomo**.

Por exemplo, em um cromossomo representado como uma sequência de números, cada número individual é um gene e determina parte do comportamento ou configuração da solução. A modificação de genes — por mutação ou recombinação — altera o cromossomo e, consequentemente, modifica o indivíduo. Assim, os genes são os “blocos de construção” das soluções, e o seu arranjo determina o desempenho de cada indivíduo dentro da população.

2.3.4 *Fitness function*

A função de aptidão (ou *fitness function*) é o mecanismo central de avaliação em um algoritmo genético. Ela **mede a qualidade de cada indivíduo da população**, atribuindo um valor que representa o quão boa é a solução representada por seu cromossomo em relação ao objetivo do problema. Esse valor é essencial para guiar o

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

processo evolutivo, pois determina quais indivíduos têm maior chance de serem selecionados para reprodução e contribuir com seus genes nas próximas gerações.

A forma exata da *fitness function* depende do problema: pode ser uma função a ser maximizada (como lucro, acurácia ou eficiência), ou transformada em um critério de maximização, mesmo em problemas de minimização (como erro ou custo). Por exemplo, em um problema de roteamento, a função de aptidão pode avaliar o tempo total do trajeto, na classificação, pode ser a taxa de acerto.

A *fitness function* deve ser cuidadosamente projetada, pois define as pressões seletivas dentro do algoritmo, se ela não for precisa ou bem calibrada, o algoritmo pode se orientar por objetivos equivocados ou convergir para soluções subótimas.

2.3.5 Conceitos fundamentais aplicados a um exemplo

As definições acima podem ter feito sentido do ponto de vista biológico, porém, como os conceitos podem ser empregados em computação para nortear ainda mais a construção de um algoritmo? Vamos lá!

Imagine que queremos encontrar a melhor combinação de hiperparâmetros que maximize a performance (exemplo: acurácia) de um modelo Random Forest em um determinado conjunto de dados. Os hiperparâmetros a serem ajustados incluem, por exemplo, os números de árvores, a profundidade máxima, o número mínimo de amostras por folha, entre outros.

Nosso objetivo é encontrar a combinação de hiperparâmetros que maximize a performance do modelo. Entretanto, nosso espaço de busca é muito amplo, por isso, métodos de força bruta seriam inviáveis e apenas “chutar” uma combinação pode não ser efetivo.

Podemos partir para a formulação como um “problema de algoritmo genético”, da seguinte forma:

- **População:** um conjunto de diferentes combinações de hiperparâmetros. Cada elemento da população representa uma configuração distinta do modelo a ser testado.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- **Indivíduo:** uma única combinação de hiperparâmetros — ou seja, uma configuração específica do Random Forest. Cada indivíduo é uma solução candidata ao problema de otimização.
- **Cromossomo:** a representação codificada da configuração de hiperparâmetros de um indivíduo. Pode ser uma lista ou vetor com os valores de cada hiperparâmetro. Por exemplo:

Cromossomo = [100, 10, 2]

Número de árvores = 100

Profundidade máxima = 10

Número mínimo de amostras por folha = 2

- **Gene:** cada hiperparâmetro individual dentro do cromossomo. No exemplo anterior, cada valor (100, 10, 2) é um gene que representa um aspecto do modelo.
- ***Fitness function*:** a função que avalia a qualidade de cada indivíduo. Neste caso, pode ser a acurácia obtida através da validação cruzada no conjunto de dados com a combinação de hiperparâmetros representada pelo indivíduo. Quanto maior a acurácia, maior a aptidão da solução.

Portanto, evoluiremos para o melhor conjunto de hiperparâmetros a partir de uma população aleatória que iria melhorando com o tempo.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3 CICLO DE EXECUÇÃO DE UM ALGORITMO GENÉTICO

3.1 Etapas básicas

O ciclo de execução de um algoritmo genético é o coração do seu funcionamento. É nesse fluxo iterativo que a população de soluções evolui, geração após geração, combinando mecanismos de variação e seleção que simulam o processo da evolução natural. O grande diferencial desse ciclo é a maneira como ele equilibra forças opostas, mas complementares: a **exploration**, que permite ao algoritmo descobrir novas regiões do espaço de busca por meio da diversidade; e a **exploitation**, que foca em refinar as melhores soluções já encontradas, buscando aperfeiçoamento contínuo. O processo está ilustrado na figura a seguir:

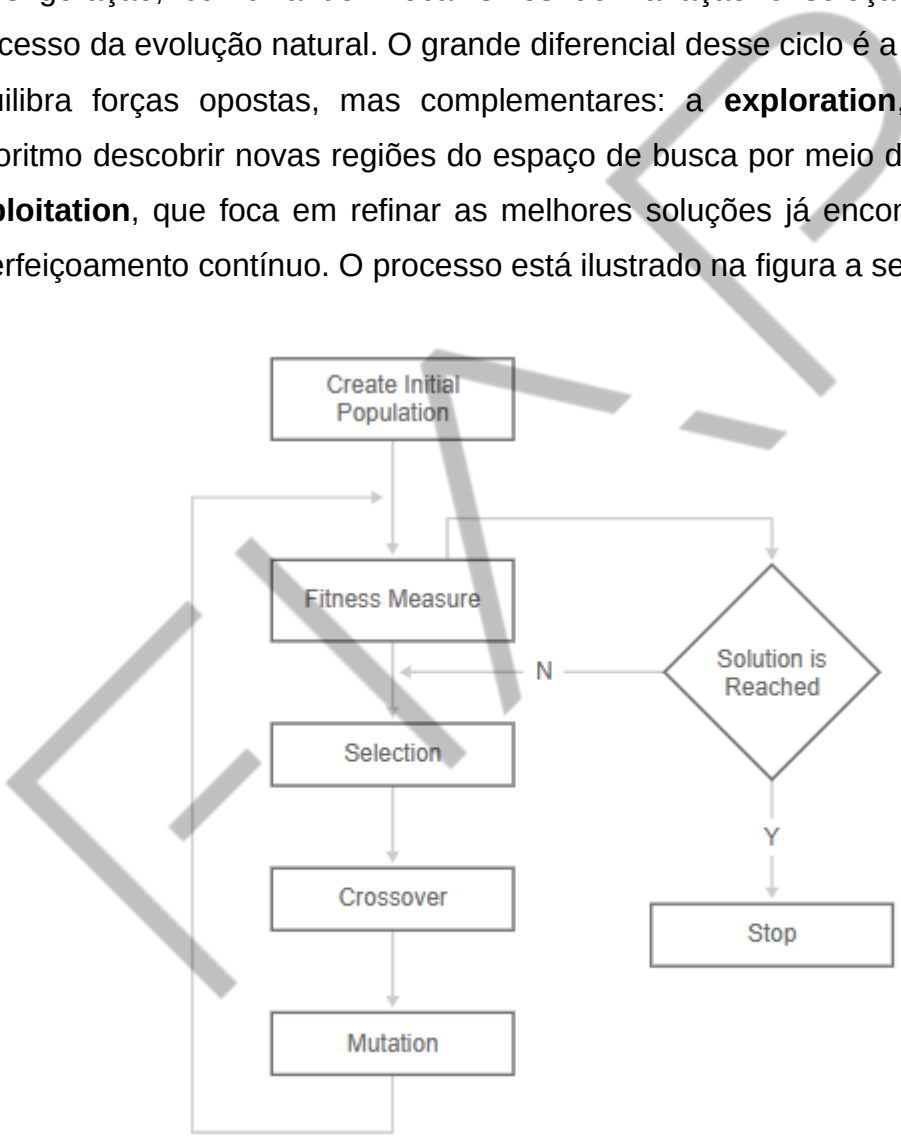


Figura 5 - Passos do algoritmo genético
Fonte: ResearchGate (2025), adaptado por FIAP (2025)

Ao longo desse ciclo, soluções são avaliadas, selecionadas com base em seu desempenho, recombinadas para formar novas possibilidades e ocasionalmente modificadas para manter a inovação. **Esse processo não é aleatório nem**

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

completamente direcionado, ele é adaptativo. Ele aprende com o desempenho das gerações anteriores, mas mantém espaço para o inesperado. Esse é o segredo da sua robustez: a capacidade de buscar o ótimo global sem se perder no caminho. A seguir, exploraremos cada uma dessas etapas em detalhe, entendendo como elas interagem para guiar o algoritmo em direção a soluções cada vez melhores.

3.1.1 Inicialização aleatória da população

A inicialização aleatória da população é a primeira etapa do ciclo de um AG e consiste em gerar um conjunto inicial de soluções candidatas (indivíduos) de forma totalmente aleatória. Cada indivíduo possui um cromossomo com genes que representam diferentes valores dos parâmetros do problema. Essa aleatoriedade é fundamental para garantir diversidade genética, permitindo que o algoritmo explore múltiplas regiões do espaço de busca desde o início, sem viés prévio. Uma população inicial bem distribuída aumenta as chances de encontrar boas soluções ao longo das gerações.

3.1.2 Avaliação de aptidão (fitness)

A avaliação de aptidão, também conhecida como cálculo da fitness, é a etapa em que cada indivíduo da população é analisado para determinar o quão boa é a solução que ele representa. Essa avaliação é realizada por meio de uma função de aptidão (*fitness function*), que atribui um valor numérico a cada indivíduo com base em seu desempenho em relação ao objetivo do problema.

Por exemplo, em um problema de classificação, a fitness pode ser a acurácia de um modelo treinado com os parâmetros representados pelo indivíduo; em um problema de roteamento, pode ser a distância total percorrida. Indivíduos com valores de fitness mais altos (em problemas de maximização) são considerados mais aptos e, por isso, têm maior probabilidade de serem selecionados para gerar a próxima geração. Essa etapa é crucial, pois guia todo o processo evolutivo, indicando quais soluções devem ser preservadas, recombinadas ou descartadas.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3.1.3 Seleção dos indivíduos mais aptos

A seleção dos indivíduos mais aptos é a etapa em que os melhores candidatos da população atual são escolhidos com base em seus valores de aptidão (fitness). O objetivo é priorizar as soluções mais promissoras para participarem da reprodução, garantindo que suas características tenham mais chances de serem passadas adiante. Essa seleção pode ser feita por diferentes métodos, como veremos adiante, e é essencial para potencializar a exploitation, focando o esforço evolutivo nas regiões mais vantajosas do espaço de busca.

3.1.4 Reprodução (crossover e mutação)

A etapa de reprodução é responsável por gerar novos indivíduos a partir dos selecionados da população atual, combinando e modificando suas informações genéticas. Ela envolve dois processos principais: o **crossover** (cruzamento), que mistura partes dos cromossomos de dois “pais” para criar um ou mais “filhos”, e a **mutação**, que altera aleatoriamente genes individuais para introduzir diversidade. Juntas, essas operações permitem ao algoritmo explorar novas regiões do espaço de busca (exploration) e refinar boas soluções já existentes (exploitation), promovendo o avanço da população em direção a soluções mais eficazes.

3.1.5 Substituição para formar nova geração

A substituição para formar uma nova geração consiste em atualizar a população atual com os novos indivíduos gerados na reprodução. Isso pode ser feito substituindo toda a população anterior ou apenas parte dela, mantendo alguns dos melhores indivíduos. Essa etapa garante a continuidade do ciclo evolutivo, permitindo que a população evolua progressivamente ao longo das gerações.

3.1.6 Critério de parada

Os critérios de parada determinam quando o algoritmo genético deve encerrar sua execução. Eles evitam que o processo continue indefinidamente e ajudam a

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

equilibrar tempo de execução e qualidade da solução. Os critérios mais comuns incluem:

- Atingir um número máximo de gerações;
- Alcançar um valor mínimo ou máximo de fitness;
- Observar estagnação (quando não há melhoria significativa por várias gerações);
- Atingir um limite de tempo.

A escolha do critério depende do problema e dos recursos disponíveis, sendo essencial para garantir eficiência e controle sobre o processo evolutivo.

3.2 Um exemplo elucidativo

Vamos retornar ao problema de maximizar a performance do modelo de Random Forest ajustando alguns hiperparâmetros. A relação deste problema com cada etapa do processo pode ser:

1. **Inicialização aleatória da população:** o algoritmo começa gerando uma população inicial de configurações de hiperparâmetros de forma aleatória. Cada indivíduo é uma combinação diferente, como:

- Indivíduo A: [n_estimators=100, max_depth=10, min_samples_leaf=2]
- Indivíduo B: [n_estimators=150, max_depth=20, min_samples_leaf=1]
- ...

Observação: essas combinações são representadas por cromossomos, e cada valor de hiperparâmetros é um gene.

2. **Avaliação de aptidão (fitness):** cada indivíduo é avaliado por meio da função de aptidão, que mede a qualidade da solução. Neste caso, é executado o modelo Random Forest com os hiperparâmetros representados pelo indivíduo, e, em seguida, é avaliada a acurácia média por validação cruzada. Por exemplo:

- Indivíduo A → Acurácia = 0.84

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- Indivíduo B → Acurácia = 0.87

3. **Seleção dos indivíduos mais aptos:** com base na fitness, os indivíduos com melhor desempenho têm maior chance de serem selecionados para reprodução. Pode-se usar métodos como torneio ou roleta para escolher os "pais" da próxima geração.

4. **Reprodução (crossover e mutação):** a reprodução gera novos indivíduos, sendo o resultado de duas etapas:

- **Crossover:** combina genes de dois pais para formar novos cromossomos.
  - Exemplo: cruzamento entre A e B → filho: [n_estimators=100, max_depth=20, min_samples_leaf=2]
- **Mutação:** modifica aleatoriamente um gene para manter diversidade.
  - Exemplo: mutação altera max_depth de 20 para 15

Essas operações equilibram exploration (com mutações) e exploitation (com cruzamentos entre bons indivíduos).

5. **Substituição para formar nova geração:** os novos indivíduos substituem parte ou toda a população anterior. Pode-se manter os melhores (elitismo) e completar o restante com os novos gerados.

6. **Critérios de parada:** o ciclo se repete até que um critério de parada seja atingido. Exemplos:

- Número máximo de gerações (exemplo: 50 iterações);
- Fitness ótimo atingido (exemplo: acurácia > 0.90);
- Estagnação (sem melhoria após N gerações);
- Limite de tempo de execução (exemplo: 30 minutos).

Ao final do processo, o algoritmo retorna o indivíduo com a maior fitness — ou seja, a melhor combinação de hiperparâmetros testada, que pode então ser usada para treinar o modelo final em produção.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

4 PONTOS DE ALEATORIEDADE

4.1 Seleção

O processo de seleção tem como objetivo escolher, entre os indivíduos da população atual, aqueles que participarão da geração de novos indivíduos. A ideia é favorecer as soluções mais aptas — aquelas que apresentam melhor desempenho segundo a função de fitness — ao mesmo tempo em que mantém uma certa diversidade genética. A aleatoriedade entra como um fator essencial para evitar que apenas os melhores sejam sempre escolhidos, o que poderia levar à convergência prematura.

Entre os métodos mais comuns de seleção, destacam-se:

- **Roleta (roleta viciada ou proporcional):** cada indivíduo recebe uma fatia proporcional ao seu valor de fitness. Quanto maior a aptidão, maior a chance de ser selecionado. É uma técnica probabilística simples, mas pode ser sensível a grandes diferenças de fitness.
- **Torneio:** um grupo aleatório de indivíduos é selecionado, e o mais apto entre eles é escolhido como “pai”. Esse processo é repetido para formar os pares. Permite controlar o grau de seletividade variando o tamanho do torneio.
- **Elitismo**: garante que os melhores indivíduos da geração atual sejam preservados na próxima, sem sofrerem alterações. Embora não seja uma técnica de seleção em si, costuma ser usada em conjunto com outras, para garantir que boas soluções não se percam.

A seleção combina **exploration** e **exploitation**, escolhendo bons candidatos, mas mantendo a chance de indivíduos menos aptos contribuírem com diversidade genética.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

4.2 Cruzamento (Crossover)

O crossover é o operador responsável por combinar informações genéticas de dois “pais” para gerar novos indivíduos, chamados de “filhos”. O objetivo é que as boas características de cada pai sejam combinadas para produzir soluções potencialmente melhores. A aleatoriedade define como essa mistura acontece, variando conforme o tipo de cruzamento adotado.

Os tipos mais comuns são:

- **Ponto único (*Single-point crossover*):** escolhe-se aleatoriamente um ponto de corte no cromossomo. Os genes antes do ponto vêm de um pai, e os genes após o ponto vêm do outro. É simples e eficaz para representações lineares. Observe a figura:

Figura 6 - Exemplo do crossover de ponto único
Fonte: Wikipedia (2025), adaptado por FIAP (2025)

- **Dois pontos (*Two-point crossover*):** define-se dois pontos aleatórios de corte, e a parte intermediária do cromossomo é trocada entre os pais. Oferecendo mais variação do que o cruzamento de ponto único. Observe:

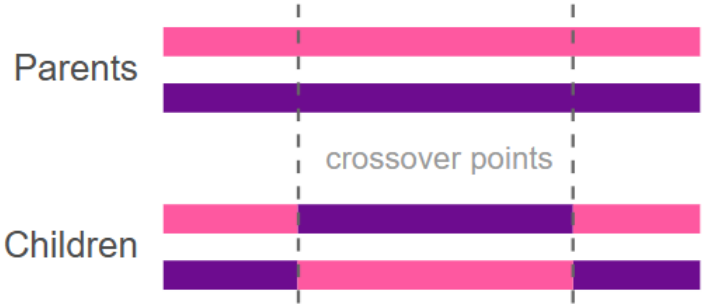


Figura 7 - Exemplo do crossover de ponto duplo
Fonte: Wikipedia (2025), adaptado por FIAP (2025)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- **Cruzamento uniforme (*Uniform crossover*):** cada gene do cromossomo é escolhido aleatoriamente de um dos pais, com igual probabilidade. Proporcionando uma maior recombinação genética, favorecendo, assim, a exploration. Nisso, basicamente jogamos uma moeda para cada cromossomo para decidir se ele será ou não incluído na prole.

O crossover é um dos principais responsáveis pela **exploitation**, pois permite explorar intensivamente regiões promissoras do espaço de soluções combinando genes de alto desempenho.

4.3 Mutação

A mutação é o operador que introduz variações aleatórias nos genes de um indivíduo, modificando parte de sua estrutura genética. Seu principal objetivo é manter a diversidade genética ao longo das gerações, evitando que a população se torne homogênea demais e o algoritmo fique preso em soluções subótimas.

Além disso, ela atua como “força de **exploration**”, ajudando a descobrir regiões do espaço de busca que não seriam alcançadas apenas com o cruzamento. A mutação pode ocorrer em:

- **Mutação de inversão:** dois genes aleatórios são escolhidos e realiza-se a inversão de todos os genes. Isso faz com que seja preservada a informação adjacente entre os genes, porém, perde-se informação de ordem. Observe a figura:

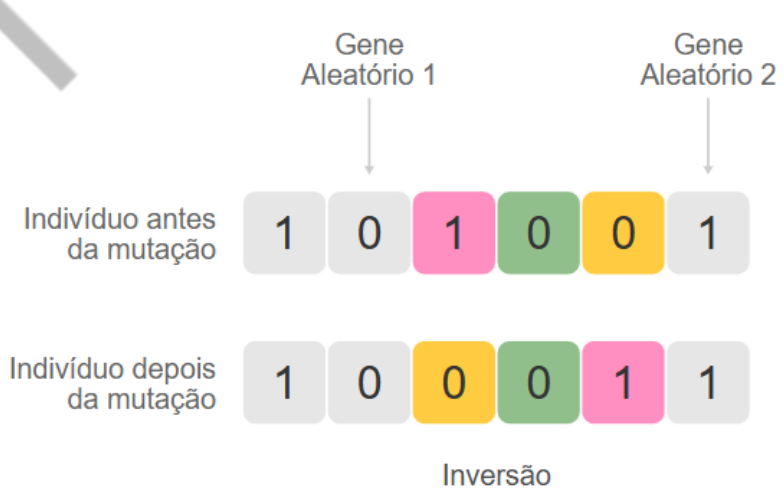


Figura 8 - Mutação de inversão
Fonte: Bioinfo (2021), adaptado por FIAP (2025)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- **Representações binárias:** inversão do valor de um bit (de 0 para 1 ou vice-versa). Observe:

Figura 9 - Mutação binária
Fonte: Bioinfo (2021), adaptado por FIAP (2025)

- **Listas ou permutações:** troca de posições entre dois elementos ou inserção/remoção aleatória.

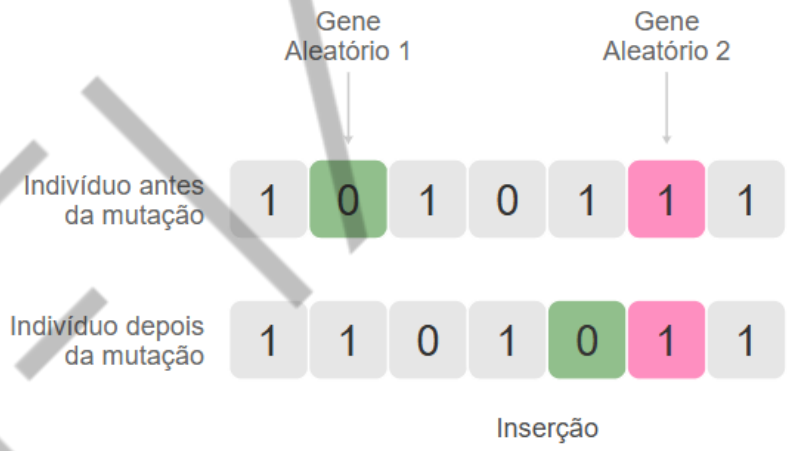


Figura 10 - Mutação por permutação
Fonte: Bioinfo (2021), adaptado por FIAP (2025)

Apesar de ser essencial para a inovação, é necessário que a mutação seja aplicada com probabilidade controlada. Taxas muito altas podem destruir boas soluções, enquanto taxas muito baixas podem levar à estagnação.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

4.4 Parâmetros

Os parâmetros do algoritmo genético controlam seu comportamento global e têm impacto direto sobre o equilíbrio entre exploration e exploitation. São definidos antes da execução e devem ser ajustados com cuidado, pois determinam a eficiência e eficácia da busca.

Entre os principais parâmetros, destacam-se:

- **Tamanho da população:** influencia a diversidade inicial e a capacidade de explorar o espaço de busca. Populações maiores oferecem mais variedade, mas aumentam o custo computacional.
- **Taxa de cruzamento (*crossover rate*):** determina a frequência com que o cruzamento será aplicado. Taxas altas favorecem o reaproveitamento de boas soluções, promovendo exploitation.
- **Taxa de mutação (*mutation rate*):** define a probabilidade de cada gene sofrer mutação. É a principal força de exploration, e deve ser cuidadosamente balanceada para manter a diversidade sem comprometer a estabilidade da população.

A escolha adequada desses parâmetros é essencial para que o AG atinja um bom desempenho. Em muitos casos, técnicas adaptativas ou autoajustáveis são empregadas para ajustar dinamicamente esses valores durante a execução.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

5 VANTAGENS E LIMITAÇÕES

Os algoritmos genéticos se destacam como uma poderosa ferramenta de otimização, especialmente em problemas complexos, mal definidos ou com múltiplos ótimos. Uma de suas grandes vantagens é a independência em relação à estrutura matemática do problema: eles não exigem o cálculo de derivadas nem qualquer conhecimento prévio do espaço de busca. Isso os torna ideais para funções “caixa-preta”, altamente não lineares, descontínuas ou com regiões de difícil modelagem. Além disso, são naturalmente adequados em espaços grandes e multidimensionais, nos quais outras técnicas de busca rapidamente se tornam inviáveis.

Outra vantagem dos AGs é a sua robustez a ruídos e a presença de múltiplos ótimos locais. Por operarem com populações e contarem com mecanismos estocásticos de exploração, conseguem manter alternativas viáveis ativas por várias gerações, o que aumenta as chances de escapar de regiões subótimas e alcançar soluções mais globais. Essa característica os torna extremamente valiosos em cenários com dados imperfeitos, simulações lentas ou funções custo instáveis.

No entanto, os algoritmos genéticos não estão isentos de limitações. Seu custo computacional pode ser elevado, especialmente quando a função de fitness é cara de calcular, como em treinamentos de modelos complexos ou simulações demoradas. Outro ponto sensível é a escolha dos parâmetros do algoritmo, como taxas de mutação e crossover, tamanho da população e critério de parada, que influenciam diretamente o desempenho e requerem ajustes cuidadosos. Além disso, se não houver mecanismos para manter a diversidade genética, o algoritmo pode sofrer convergência prematura, ou seja, fixar-se rapidamente em uma solução ruim sem explorar outras possibilidades.

Em comparação com métodos determinísticos, como o gradiente descendente, os algoritmos genéticos oferecem maior flexibilidade e adaptabilidade, porém, muitas vezes são menos eficientes quando o problema possui uma estrutura matemática bem definida e explorável com métodos clássicos. Em resumo, os AGs são mais indicados para terrenos acidentados, desconhecidos ou imprevisíveis, mas que exigem cuidado e estratégia para explorar todo o seu potencial evolutivo.

As vantagens e limitações estão listadas no quadro a seguir:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Categoria	Comentários
Vantagens	Não exigem derivadas ou conhecimento prévio do problema.
	Adaptam-se bem a espaços grandes e não lineares.
	Robustos a ruídos e múltiplos ótimos locais.
Limitações	Alto custo computacional, especialmente com fitness caro.
	Sensíveis à escolha dos parâmetros.
	Risco de convergência prematura sem diversidade.
Comparações	Estocásticos, ao contrário de métodos determinísticos.
	Mais flexíveis que gradientes, porém, menos eficientes em funções bem definidas.

Quadro 1 - Vantagens e limitações dos algoritmos genéticos
Fonte: Elaborado pelo autor (2025)

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

6 HANDS ON: PRATICANDO ALGORITMOS GENÉTICOS

Agora que conhecemos os aspectos centrais dos algoritmos genéticos, está na hora de praticarmos sua implementação. Nossa prática será feita em um jupyter notebook no ambiente do Google Colaboratory (Colab).

Dica: veja o material HOW TO para acessar o ambiente do Colab.

Uma vez no Colab, crie um novo notebook, dê um nome como preferir (sugestão: Aula 6), e faça cada bloco de código-fonte em uma célula. Após inserir o código na célula do notebook, execute-a e veja a saída gerada.

Observação: para o código deste capítulo, não temos arquivo de apoio. No entanto, como a geração da entrada envolve números aleatórios, pode ser que os seus resultados sejam diferentes do material, mas seguindo o formato apresentado.

Sobre a o problema:

Nesta atividade, trabalharemos com o problema da mochila binária (ou 0-1 *knapsack problem*), um clássico da otimização combinatória e que pode ser descrito da seguinte forma: dado um conjunto de itens, cada um com um peso e um valor, o objetivo é selecionar um subconjunto desses itens de modo que a soma dos valores seja maximizada, sem que a soma dos pesos ultrapasse a capacidade máxima da mochila. Observe a figura:

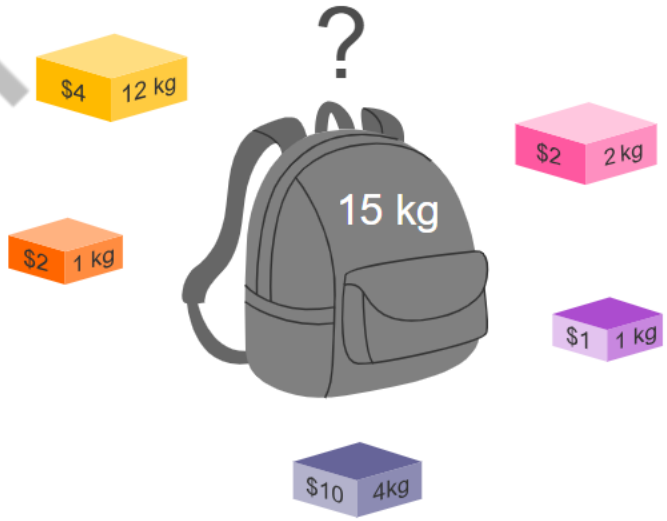


Figura 11 - Ilustração do problema da mochila binária
Fonte: Wikimedia (2020), adaptado por FIAP (2025)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

A versão binária do problema impõe que cada item só pode ser totalmente incluído (1) ou totalmente excluído (0) — ou seja, os itens não podem ser fracionados. Cada solução possível pode ser representada por um vetor binário, em que o valor 1 indica que o item foi escolhido e o valor 0 que ele foi descartado.

Esse problema é considerado NP-difícil, ou seja, não existe um algoritmo conhecido que o resolva de forma eficiente para grandes conjuntos de itens. Por isso, é frequentemente utilizado como benchmark para testar métodos heurísticos e metaheurísticos, como algoritmos genéticos, que conseguem encontrar boas soluções em tempo viável mesmo para instâncias complexas.

Vamos para o código!

Começaremos com os imports pertinentes, a modelagem do problema e a exibição dos dados:

```
import numpy as np
import pandas as pd
import random as rd
from random import randint
import matplotlib.pyplot as plt

item_number = np.arange(1,41)                    #itens que
podem ir para a mochila
weight = np.random.randint(1, 15, size=40)  # pesos
aleatorios
value = np.random.randint(1, 100, size=40)  # valores
aleatorios
knapsack_capacity = 100                       # peso máximo
da mochila

print('Capacidade da mochila: ', knapsack_capacity)
print('Possíveis itens:')
print('Item\tPeso\tValor')
for i in range(len(item_number)):
  print(f"{item_number[i]}\t{weight[i]}\t{value[i]}")
```

Código-fonte 1 - Imports e carregamento de dados
Fonte: Elaborado pelo autor (2025)

A saída está ilustrada na figura “Saída do Código-fonte 1”.

Importante: pode ser que sua saída seja diferente, pois a modelagem parte de aleatorização. O importante é determinarmos o número de itens, seus pesos, valores e a capacidade da mochila.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
Capacidade da mochila:  100
Possíveis itens:
Item    Peso    Valor
1       9       6
2       3       59
3       2       88
4       10      77
5       4       39
6       9       73
7       7       53
8       2       93
9       9       27
10      12      17
11      10      38
12      7       40
13      5       78
14      6       4
15      14      73
16      7       40
17      8       2
18      6       73
19      12      45
20      5       77
21      7       55
22      12      10
23      4       93
24      3       1
25      6       22
26      4       25
27      3       40
28      9       77
29      2       21
30      13      8
31      13      55
32      12      47
33      8       44
34      9       51
35      4       20
36      1       93
37      7       60
38      4       94
39      2       49
40      13      25
```

Figura 12 - Saída do Código-fonte 1
Fonte: Elaborado pelo autor (2025)

Vamos gerar a população inicial. Para isso, definimos que a população inicial deve ter 16 indivíduos, e será aprimorada por 100 gerações:

```
solutions_per_pop = 16
pop_size = (solutions_per_pop, len(item_number))
print(f'Tamanho da população = {pop_size}')
initial_population = np.random.randint(2, size = pop_size)
initial_population = initial_population.astype(int)
num_generations = 1000
print(f'População inicial:\n {initial_population}')
```

Código-fonte 2 - Gerando a população
Fonte: Elaborado pelo autor (2025)

Sua saída deve ser parecida (mas não necessariamente idêntica) à exibida na figura “Saída do Código-fonte 2”. Ela indica que temos uma população inicial de 16 indivíduos, cada um com tamanho 40 (pois temos no máximo 40 itens à disposição

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

para pôr na mochila). Cada lista de 0s e 1s é um cromossomo, e cada 0 ou cada 1 é um gene:

```
Tamanho da população = (16, 40)
População inicial:
 [[0 0 0 0 0 0 0 1 1 0 1 0 1 0 1 0 1 0 1 1 1 0 0 1 0 1 0 1 0 0 1 0 0 0 0 0 0
  1 0 1 0]
 [0 0 1 0 0 0 1 1 1 1 1 0 1 0 1 0 0 1 1 1 1 0 0 0 1 1 1 0 0 0 0 0 1 1 1 1
  1 1 0 0]
 [1 0 1 0 1 1 1 0 0 1 1 0 1 1 0 1 1 1 0 1 0 1 0 1 1 0 1 0 1 1 0 0 0 0 1 0
  0 1 1 1]
 [0 1 0 1 0 0 0 0 1 0 0 1 0 1 0 1 0 1 0 0 1 1 0 0 1 0 1 1 1 1 0 0 0 1 0 0
  0 0 1 1]
 [0 0 0 1 1 0 0 1 1 1 1 0 1 1 0 0 0 0 1 0 1 0 1 1 0 0 0 1 1 0 1 0 1 1 0 1
  1 0 1 0]
 [0 1 0 0 0 0 0 0 1 0 1 0 0 1 0 0 0 0 0 0 1 1 1 1 1 1 1 0 0 0 0 1 1 0 1 0
  0 1 1 0]
 [1 1 1 1 0 1 1 1 1 1 1 1 1 1 1 1 1 0 0 0 1 0 1 1 1 1 0 1 0 1 0 1 1 0 0 1
  1 0 0 1]
 [0 1 1 0 1 1 1 1 1 0 1 1 1 0 1 0 1 1 0 0 0 0 0 1 0 0 0 1 0 0 1 0 0 1 0 1
  0 1 1 1]
 [0 0 1 0 1 0 1 0 0 0 1 0 0 0 0 1 1 1 1 0 1 1 1 1 0 0 1 0 1 1 0 0 0 1 0 0
  1 0 0 0]
 [1 0 0 1 1 0 1 1 1 1 0 1 1 1 1 1 0 1 0 0 0 0 1 1 1 1 1 0 0 0 0 0 1 1 1 0
  0 1 1 0]
 [1 1 1 1 1 1 0 1 0 0 0 0 0 1 1 0 1 0 0 1 1 1 1 1 1 1 1 0 1 0 0 1 1 1 1 0
  1 0 1 0]
 [0 1 0 0 1 0 0 1 0 1 1 1 0 1 1 0 1 1 1 1 1 0 0 0 0 0 1 1 0 1 1 1 1 0 0 0
  1 0 0 0]
 [0 0 1 1 1 0 1 1 1 0 0 0 1 1 0 1 0 1 0 1 1 1 1 0 0 1 1 0 0 0 0 0 1 1 0 1
  1 0 1 1]
 [1 1 1 0 1 1 0 1 0 1 0 1 0 1 1 0 0 1 1 0 0 0 1 1 1 0 1 0 0 0 0 0 1 1 0 1
  1 0 0 1]
 [1 1 1 1 1 0 1 0 0 0 0 1 0 1 1 1 1 0 1 1 0 1 1 0 0 0 0 1 1 1 0 1 0 1 1 1
  1 1 1 1]
 [0 1 0 1 1 0 0 0 0 0 0 1 0 0 1 1 0 0 0 1 1 1 0 0 0 1 0 1 0 1 0 1 0 0 1 0 1
  0 1 0 0]]
```

Figura 13 - Saída do Código-fonte 2
Fonte: Elaborado pelo autor (2025)

Feito isso, podemos definir as **funções de fitness** (que avalia se a solução cabe na mochila e qual é seu valor associado), **seleção** (definir os indivíduos da nova geração), **crossover** (combinação de pais para formar descendentes) e **mutação** (outro ponto de aleatoriedade):

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
# AVALIAÇÃO DO FITNESS DA SOLUÇÃO
def fitness(weight, value, population, capacity):
  fitness = np.empty(len(population))
  for i in range(len(fitness)):
    S1 = np.sum(population[i] * value)
    S2 = np.sum(population[i] * weight)
    if S2 <= capacity:
      fitness[i] = S1
    else:
      fitness[i] = 0
  return fitness.astype(int)


# SELEÇÃO DE INDIVÍDUOS PARA A PRÓXIMA GERAÇÃO
def selection(fitness, num_parents, population):
  fitness = list(fitness)
  parents = np.empty((num_parents, population.shape[1]))
  for i in range(num_parents):
          max_fitness_idx    =    np.where(fitness    ==
np.max(fitness))
    parents[i,:] = population[max_fitness_idx[0][0], :]
    fitness[max_fitness_idx[0][0]] = -9999
  return parents

# CRUZAMENTO DE PAIS PARA GERAR FILHOS
def crossover(parents, num_offsprings):
        offsprings      =      np.empty((num_offsprings,
parents.shape[1]))
  crossover_point = int(parents.shape[1]/2)
  crossover_rate = 0.8
  # i representa o indice dos pais
  i = 0
  # cnt_offsprings = numero de filhos gerados
  cnt_offsprings = 0

  while (cnt_offsprings < num_offsprings):
    parent1_index = i%parents.shape[0] #index do genitor
1
     parent2_index  =  (i+1)%parents.shape[0]  #indice  do
genitor 2
    x = rd.random()
    if x > crossover_rate:
      pass
    else:
          offsprings[cnt_offsprings,0:crossover_point]  =
parents[parent1_index,0:crossover_point]
           offsprings[cnt_offsprings,crossover_point:]  =
parents[parent2_index,crossover_point:]
      cnt_offsprings +=1
    i += 1
  return offsprings
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
# MUTAÇÃO DE GENES DE CROMOSSOMO
def mutation(offsprings):
    mutants = np.empty((offsprings.shape))
    mutation_rate = 0.15
    for i in range(mutants.shape[0]):
        random_value = rd.random()
        mutants[i,:] = offsprings[i,:]
        if random_value > mutation_rate:
            continue
        int_random_value = randint(0,offsprings.shape[1]-
1)
        if mutants[i,int_random_value] == 0 :
            mutants[i,int_random_value] = 1
        else :
            mutants[i,int_random_value] = 0
    return mutants
```

Código-fonte 3 - Definição de funções estruturantes do algoritmo genético
Fonte: Elaborado pelo autor (2025)

A célula anterior não gera saída, mas é extremamente relevante no nosso código, pois define as funções estruturantes do algoritmo genético. Note que tal código só faz sentido no contexto do problema da mochila binária. Outros problemas precisam ser modelados de outra forma, potencialmente com entradas e saídas diferentes. O código só faz sentido da forma como apresentado, pois optamos por representar a população como uma lista de listas. Alterações nas estruturas de dados implicariam em alterações nos códigos.

Chegou a hora de definir a função principal, aquela que amarra as etapas do algoritmo genético! A estrutura é simples:

- A partir de uma população inicial;
- Enquanto puder criar gerações:
  - Avalie o fitness dos indivíduos;
  - Selecione quem entrará na próxima geração;
  - Faça descendentes;
  - Crie mutações.
- Ao final, avalie o fitness da população resultante e retorne o indivíduo com o melhor resultado para o problema.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
def optimize(weight, value, population, pop_size, num_generations, capacity):
        parameters, fitness_history = [], []
        num_parents = int(pop_size[0]/2)
        num_offsprings = pop_size[0] - num_parents
        for i in range(num_generations):
              fitness_ = fitness(weight, value, population, capacity)
            fitness_history.append(fitness_)
                parents = selection(fitness_, num_parents, population)
            offsprings = crossover(parents, num_offsprings)
            mutants = mutation(offsprings)
            population[0:parents.shape[0], :] = parents
            population[parents.shape[0]:, :] = mutants

        print('Última geraçào: \n{}\n'.format(population))
        fitness_last_gen = fitness(weight, value, population, capacity)
                print('Fitness da última geração: \n{}\n'.format(fitness_last_gen))
            max_fitness = np.where(fitness_last_gen == np.max(fitness_last_gen))
        parameters.append(population[max_fitness[0][0],:])
        return parameters, fitness_history
```

Código-fonte 4 - Função principal do algoritmo genético
Fonte: Elaborado pelo autor (2025)

O código anterior também não possui saída, mas seu entendimento é relevante! Chegou a hora de evocá-lo, efetivamente, para vermos a resolução do problema com algoritmos genéticos:

```
parameters, fitness_history = optimize(weight,
                                       value,
                                       initial_population,
                                       pop_size,
                                       num_generations,
                                       knapsack_capacity)
```

Código-fonte 5 - Executando nosso algoritmo genético
Fonte: Elaborado pelo autor (2025)

Você consegue adivinhar os valores de cada parâmetro? Vamos lá!

- **`weight`**: lista de pesos dos itens;
- **`value`**: lista dos valores dos itens;
- **`initial_population`**: população inicial, com 16 indivíduos;

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- **`pop_size`**: 16 indivíduos, cada 1 com tamanho 40;
- **`num_generations`**: número de gerações permitido (1000);
- **`knapsack_capacity`**: capacidade da mochila (100).

A saída está ilustrada na figura a seguir.

Não esqueça que a sua saída pode ser diferente desta devido aos aspectos aleatórios do problema!

```
Última geração:
[[0 1 1 0 0 0 0 0 1 0 1 0 0 1 1 0 0 0 0 0 1 0 0 0 1 1 1 0 1 0 0 0 1 1 1 1
  1 1 0 0]
 [0 1 1 0 0 0 0 0 1 0 1 0 0 1 1 0 0 0 0 0 1 0 0 0 1 1 1 0 1 0 0 0 1 1 1 1
  1 1 0 0]
 [0 1 1 0 0 0 0 0 1 0 1 0 0 1 1 0 0 0 0 0 1 0 0 0 1 1 1 0 1 0 0 0 1 1 1 1
  1 1 0 0]
 [0 1 1 0 0 0 0 0 1 0 1 0 0 1 1 0 0 0 0 0 1 0 0 0 1 1 1 0 1 0 0 0 1 1 1 1
  1 1 0 0]
 [0 1 1 0 0 0 0 0 1 0 1 0 0 1 1 0 0 0 0 0 1 0 0 0 1 1 1 0 1 0 0 0 1 1 1 1
  1 1 0 0]
 [0 1 1 0 0 0 0 0 1 0 1 0 0 1 1 0 0 0 0 0 1 0 0 0 1 1 1 0 1 0 0 0 1 1 1 1
  1 1 0 0]
 [0 1 1 0 0 0 0 0 1 0 1 0 0 1 1 0 0 0 0 0 1 0 0 0 1 1 1 0 1 0 0 0 1 1 1 1
  1 1 0 0]
 [0 1 1 0 0 0 0 0 1 0 1 0 0 1 1 0 0 0 0 0 1 0 0 0 1 1 1 0 1 0 0 0 1 1 1 1
  1 1 0 0]
 [0 1 1 0 0 0 0 0 1 0 1 0 0 1 1 0 0 0 0 0 1 0 0 0 1 1 1 0 1 0 0 0 1 1 1 1
  1 1 0 0]
 [0 1 1 0 0 0 0 0 1 0 1 0 0 1 1 0 0 0 0 0 1 0 0 0 1 1 1 0 1 0 0 0 1 1 1 1
  1 1 0 0]
 [0 1 1 0 0 0 0 0 1 0 1 0 0 1 1 0 0 0 0 0 1 0 0 0 1 1 1 0 1 0 0 0 1 1 1 1
  1 1 0 0]
 [0 1 1 0 0 0 0 0 1 0 1 0 0 1 1 0 0 0 0 0 1 0 0 0 1 1 1 0 1 0 0 0 1 1 1 1
  1 1 0 0]
 [0 1 1 0 0 0 0 0 1 0 1 0 0 1 1 0 0 0 0 0 1 0 0 0 1 1 1 0 1 0 0 0 1 1 1 1
  1 1 0 0]
 [0 1 1 0 0 0 0 0 1 0 1 0 0 1 1 0 0 0 0 0 1 0 0 0 1 1 1 0 1 0 0 0 1 1 1 1
  1 1 0 0]
 [0 1 1 0 0 0 0 0 1 0 1 0 0 1 1 0 0 0 0 0 1 0 0 0 1 1 1 0 1 0 0 0 1 1 1 1
  1 1 0 0]
 [0 1 1 0 0 0 0 0 1 0 1 0 0 1 1 0 0 0 0 0 1 0 0 0 1 1 1 0 1 0 0 0 1 1 1 1
  1 1 0 0]]

Fitness da última geração:
[814 814 814 814 814 814 814 814 814 814 814 814 814 814 814 814]
```

Figura 14 - Saída do Código-fonte 5
Fonte: Elaborado pelo autor (2025)

Podemos também verificar qual foi a solução gerada, seu resultado para a mochila e quais itens foram selecionados:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
    print('Indivíduo da última geração com maior fitness:
\n{}'.format(parameters))

    selected_items = item_number * parameters

    print('\nQuais itens irão na mochila:')
    peso = 0
    for i in range(selected_items.shape[1]):

      if selected_items[0][i] != 0:
         print('{}'.format(selected_items[0][i]), end='\t')
         peso = peso + weight[i]

    print('\nPeso total da mochila = ', peso)
```

Código-fonte 6 - Exibindo o melhor resultado encontrado
Fonte: Elaborado pelo autor (2025)

A próxima figura mostra que a melhor solução encontrada para os itens em questão foi um conjunto de itens que soma 99 “pesos”, sendo que a capacidade máxima é 100. Portanto, está dentro do esperado.

```
Indivíduo da última geração com maior fitness:
[array([0, 1, 1, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1, 0,
       0, 0, 1, 1, 1, 0, 1, 0, 0, 0, 1, 1, 1, 1, 1, 1, 0, 0])]

Quais itens irão na mochila:
2       3       9       11      14      15      21      25      26      27      29      33      34      35      36      37      38
Peso total da mochila =  99
```

Figura 15 - Saída do Código-fonte 6
Fonte: Elaborado pelo autor (2025)

Inclusive, podemos aproveitar que salvamos o histórico das soluções, geração a geração, para plotar um gráfico que mostra o fitness médio da população em relação ao tempo, bem como o destaque do fitness máximo:

```
    plt.figure(figsize=(20,10))
    fitness_history_mean = [np.mean(fitness) for fitness in
fitness_history]
    fitness_history_max = [np.max(fitness) for fitness in
fitness_history]
    plt.plot(list(range(num_generations)),
fitness_history_mean, label = 'Fitness Médio')
    plt.plot(list(range(num_generations)),
fitness_history_max, label = 'Fitness Máximo')
    plt.legend()
    plt.title('Fitness através das gerações')
    plt.xlabel('Gerações')
    plt.ylabel('Fitness')
    plt.show()
    print(np.asarray(fitness_history).shape)
```

Código-fonte 7 - Evolução do resultado ao longo das gerações
Fonte: Elaborado pelo autor (2025)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

O gráfico resultante está na figura “Saída do Código-fonte 7”. Podemos ver que o algoritmo genético converge rapidamente para a solução ótima, e depois fica o resto das iterações tentando aprimorar o resultado, sem sucesso (pois já estava diante da melhor solução).

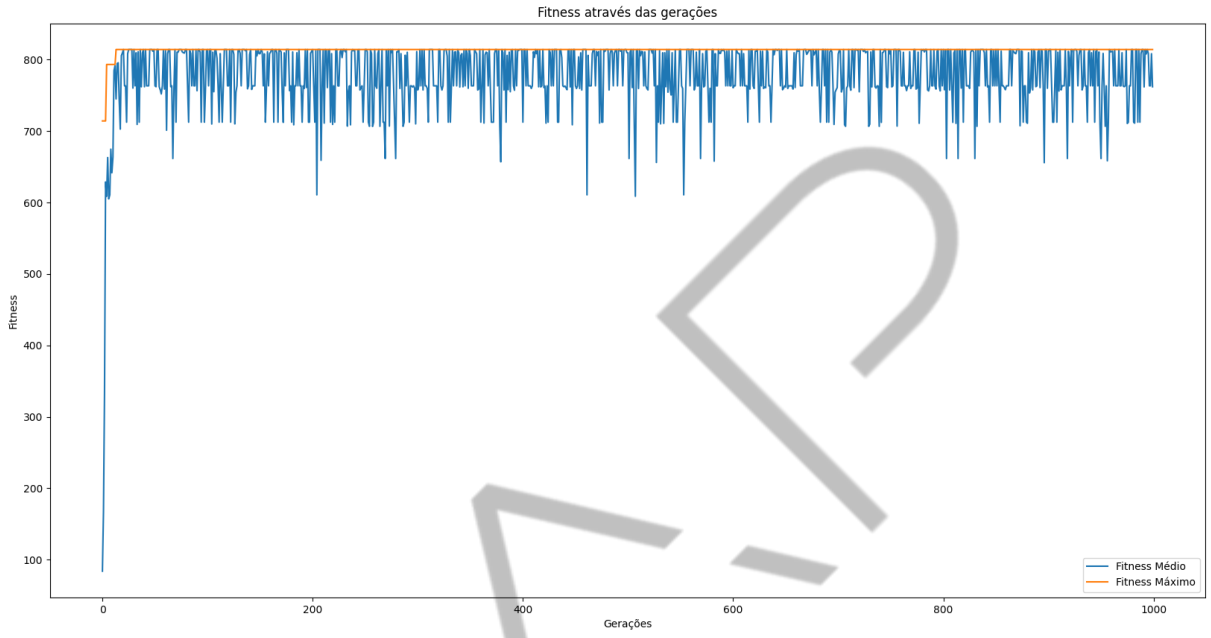


Figura 16 - Saída do Código-fonte 7
Fonte: Elaborado pelo autor (2025)

Dessa forma, compreendemos como implementar uma solução genética para o problema da mochila binária. Sabemos que a estrutura do algoritmo genético segue uma estratégia de exploitation (sendo fixa), mas que as funções de apoio conferem exploration (variabilidade). Além dos pontos de busca local, vale reforçar que cada problema possui sua implementação das funções de apoio.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

CONCLUSÃO

Os algoritmos genéticos representam uma abordagem inspirada na natureza evolutiva, que combina princípios como seleção, cruzamento e mutação para resolver problemas de forma adaptativa e inteligente. Ao longo deste capítulo, vimos como esses operadores trabalham em conjunto dentro de um ciclo evolutivo, impulsionado por uma população de soluções que se transforma geração após geração. Destacamos ainda a importância de manter o equilíbrio entre exploration, que permite a descoberta de novas possibilidades, e exploitation, que refina as melhores soluções encontradas até então.

Esses algoritmos mostram-se especialmente eficazes para solucionar problemas complexos e mal estruturados, nos quais métodos tradicionais tendem a falhar — como espaços de busca grandes, não lineares, com múltiplos ótimos ou funções custo sem derivadas acessíveis. Em cenários como ajuste de hiperparâmetros, engenharia de projetos, logística, bioinformática e até arte generativa, os AGs oferecem uma alternativa poderosa e flexível.

Mais do que uma técnica de otimização, os AGs convidam à criatividade computacional. Por isso, encorajamos você a aplicar esse conhecimento de forma prática, testando soluções para seus próprios problemas e explorando combinações inovadoras. A capacidade adaptativa e exploratória dos algoritmos genéticos pode ser exatamente o que falta para resolver desafios que pareciam intratáveis — e abrir caminho para descobertas surpreendentes.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

REFERÊNCIAS

BARNOUD, A. Imagerie géophysique de l'île volcanique de Basse-Terre en Guadeloupe, par inversion de données sismologiques, gravimétriques et magnétiques. **Géophysique**, 2016. Disponível em: <https://theses.hal.science/tel-01529757/document>. Acesso em: 31 mar. 2025.

CROSSOVER (evolutionary algorithm). **Wikipedia**, 17 de março de 2025. Disponível em: <https://en.wikipedia.org/wiki/Crossover_(evolutionary_algorithm)>. Acesso em: 31 mar. 2025.

DARA, Maher; EZZAT, M.; ABDEL-SALAM, Tarek S. Flowchart of optimization with Genetic Algorithm, B Partial Swarm Optimization. **ResearchGate**, maio de 2020. Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International. Disponível em: <https://www.researchgate.net/figure/flowchart-of-optimization-with-Genetic-Algorithm-B-Partial-Swarm-Optimization-This_fig1_363766244>. Acesso em: 31 mar. 2025.

DE LACERDA, Estéfane G. M; DE CARVALHO, A. C. P. L. F. Introdução aos algoritmos genéticos. **Sistemas inteligentes: aplicaçoes a recursos hıdricos e ciências ambientais**, 1999, 1: 99-148.

FILE:Knapsack.svg. **Wikimedia Commons**, 13 de setembro de 2020. Disponível em: <https://commons.wikimedia.org/wiki/File:Knapsack.svg>. Acesso em: 31 mar. 2025.

HENRIQUE, Pedro. O que são algoritmos genéticos e para que servem? **Deviante**, 7 de maio de 2021. Disponível em: <https://www.deviante.com.br/noticias/o-que-sao-algoritmos-geneticos-e-para-que-servem/>. Acesso em: 31 mar. 2025.

KATO, Rodrigo. Algoritmos Genéticos. **Bioinfo**, 09 de abril de 2021. Disponível em: <https://bioinfo.com.br/algoritmos-geneticos/>. Acesso em: 31 mar. 2025.

KATOCH, Sourabh; CHAUHAN, Sumit Singh; KUMAR, Vijay. A review on genetic algorithm: past, present, and future. **Multimedia tools and applications**, 2021, 80: 8091-8126.

LINDEN, Ricardo. **Algoritmos Genéticos** (3ª edição). 2012.

OTOVIĆ, Erik, *et al.* **Genetic Algorithm Parametrization for Informed Exploration of Short Peptides Chemical Space**. In: 2020 International Conference on Software, Telecommunications and Computer Networks (SoftCOM). IEEE, 2020. p. 1-3.

STANDARD Genetic Algorithm (SGA). **XLoptimizer**, 2025. Disponível em: <https://xloptimizer.com/features/genetic-algorithms-ga/standard-genetic-algorithm>. Acesso em: 31 mar. 2025.

UMBARKAR, Anant J.; SHETH, Pranali D. Crossover operators in genetic algorithms: a review. **ICTACT journal on soft computing**, 2015, 6.1.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

GLOSSÁRIO

Termo	Explicação
AG	Algoritmo Genético.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com